

Transfer Learning CNN

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

Transfer Learning in Keras (CNN) — সম্পূর্ণ বাংলা নোট

বিষয়: Transfer Learning কী, কীভাবে কাজ করে, Feature Extraction vs Fine-Tuning, এবং Keras-এ VGG16, ResNet50, MobileNetV2 দিয়ে Implementation।

১. সহজ ভাষায় পরিচিতি (Intuition)

Transfer Learning কী?

কল্পনা করো তুমি একজন অভিজ্ঞ রাঁধুনি যিনি ১০ বছর ধরে বাংলাদেশি রান্না করছেন। এখন তোমাকে বলা হলো ইটালিয়ান পিৎজা বানাতে। তুমি কি একদম শূন্য থেকে শুরু করবে? না! তুমি তোমার ইতিমধ্যে শেখা দক্ষতাগুলো (আঁচ নিয়ন্ত্রণ, উপকরণ মেশানো, সময় হিসাব) সরাসরি কাজে লাগাবে এবং শুধু পিৎজার নতুন কৌশলটুকু শিখবে।

Transfer Learning হলো ঠিক এটাই!

একটি Neural Network যে একটি বড় কাজে (যেমন ১৪০ লাখ ছবি দিয়ে ১০০০টি বস্তু চেনা — ImageNet) শিখেছে, সেই শেখা জ্ঞান নতুন, ছোট কাজে (যেমন কুকুর vs বিড়াল চেনা) কাজে লাগানো।

আরও সহজ উদাহরণ:

বাস্তব জীবন	Machine Learning
অভিজ্ঞ ডাক্তার নতুন রোগ শেখে	Pretrained Model নতুন class শেখে
ফুটবল খেলোয়াড় হকি শেখে	ImageNet Model নতুন dataset-এ train হয়
বাংলা জানলে সংস্কৃত সহজ হয়	ছবি feature-এর জ্ঞান transferable

কেন দরকার? কোন সমস্যা সমাধান করে?

সমস্যা: Deep Learning-এর জন্য দরকার:

- X লক্ষ লক্ষ labeled ছবি
- X কয়েক সপ্তাহের GPU training
- X হাজার হাজার ডলারের compute cost

Transfer Learning সমাধান দেয়:

- ☒ মাত্র কয়েকশো ছবি দিয়েই ভালো accuracy
- ☒ কয়েক মিনিট বা ঘণ্টায় training শেষ
- ☒ সাধারণ GPU বা এমনকি CPU-তেও চলে

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

CNN কীভাবে শেখে?

একটি CNN (Convolutional Neural Network) ছবি দেখলে স্তরে স্তরে feature শেখে:

প্রথম স্তর → edges, lines, curves (মৌলিক আকার)
 দ্বিতীয় স্তর → corners, textures (বুনট)
 তৃতীয় স্তর → patterns, shapes (নকশা)
 চতুর্থ স্তর → object parts (চোখ, চাকা, পাখা)
 শেষ স্তর → complete objects (বিড়াল, গাড়ি, বিমান)

এই শেখা feature গুলো **প্রায় সব ধরনের ছবির জন্য** কাজে লাগে! তাই ImageNet-এ train করা model-এর early layers-এর weight গুলো নতুন কাজেও কার্যকর।

Transfer Learning-এর দুটি প্রধান পদ্ধতি:

পদ্ধতি ১: Feature Extraction (ফিচার নিষ্কাশন)

- Pretrained model-এর সব layer **Freeze** করা হয় (weights আর update হয় না)
- শুধু শেষে নতুন classification head যোগ করা হয়
- শুধু সেই নতুন head-এর weight train হয়

কখন ব্যবহার করবে?

- Dataset ছোট (কয়েকশো থেকে কয়েক হাজার ছবি)
- নতুন dataset-এর ছবি ImageNet-এর মতোই (everyday objects)
- Compute আর সময় কম

পদ্ধতি ২: Fine-Tuning (সূক্ষ্ম সমন্বয়)

- প্রথমে Feature Extraction করো (head train করো)
- তারপর base model-এর শেষ কয়েকটা layer **Unfreeze** করো
- পুরো model খুব **ছোট learning rate** দিয়ে আবার train করো

কখন ব্যবহার করবে?

- Dataset মাঝারি থেকে বড়
- নতুন dataset ImageNet থেকে খুব আলাদা (যেমন medical images, satellite images)
- ভালো accuracy দরকার এবং সময়/GPU আছে

বিখ্যাত Pretrained Models:

Model	Parameter	Top-1 Accuracy	বৈশিষ্ট্য
VGG16	138M	71.3%	সহজ কিন্তু বিশাল
ResNet50	25M	74.9%	Skip Connection, দ্রুত
MobileNetV2	3.4M	71.8%	হালকা, মোবাইল-বান্ধব
InceptionV3	23.8M	77.9%	Multi-scale features
EfficientNetB0	5.3M	77.1%	সেরা accuracy/size ratio

Catastrophic Forgetting (বিপর্যয়কর ভুলে যাওয়া):

Fine-tuning-এর সময় একটা বড় সমস্যা! যদি learning rate বেশি হয়, model তার পুরনো শেখা (ImageNet features) ভুলে যায় এবং নতুন data-তে overfit করে ফেলে। এই কারণে fine-tuning-এ সবসময় **খুব ছোট learning rate** ($1e-5$ বা $1e-6$) ব্যবহার করতে হয়।

৩. Math / Theory

Transfer Learning-এর গাণিতিক ভিত্তি:

Source Domain: ImageNet dataset থেকে শেখা

Target Domain: তোমার নতুন dataset

Weight Initialization as Prior Knowledge (পূর্বজ্ঞান):

Scratch থেকে train করলে:

$$\theta_{\text{initial}} = \text{random}() \rightarrow \theta_{\text{optimal}} \quad [\text{অনেক দূরের যাত্রা}]$$

Transfer Learning-এ:

$$\theta_{\text{initial}} = \theta_{\text{ImageNet}} \rightarrow \theta_{\text{optimal}} \quad [\text{কাছ থেকে শুরু}]$$

যেখানে θ হলো model-এর weights।

Feature Extraction-এর Math:

$$y_{\text{target}} = g(f(x; \theta_{\text{frozen}}) ; \theta_{\text{new}})$$

- x = input image
- $f(\cdot)$ = frozen pretrained backbone (base model)
- θ_{frozen} = frozen weights (update হয় না)
- $g(\cdot)$ = নতুন classification head
- θ_{new} = নতুন layer-এর trainable weights
- y_{target} = predicted class

Fine-Tuning-এর Loss Function:

$$L_{\text{total}} = L_{\text{CE}}(y_{\text{pred}}, y_{\text{true}})$$

যেখানে Categorical Cross-Entropy:

$$L_{\text{CE}} = -\sum y_{\text{true}} * \log(y_{\text{pred}})$$

Fine-tuning-এ weight update:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta * \partial L / \partial \theta_{\text{old}}$$

- η = learning rate (fine-tuning-এ খুব ছোট, যেমন $1e-5$)
- $\partial L / \partial \theta$ = gradient (backpropagation দিয়ে calculate)

কতটুকু Layer Unfreeze করবে?

সাধারণ নিয়ম—Dataset-এর সাথে Source-এর মিলের উপর নির্ভর করে:

Dataset ছোট + Similar domain → শুধু নতুন Head train করো
 Dataset ছোট + Different domain → কিছু আগের layer train করো
 Dataset বড় + Similar domain → Fine-tune শেষ ১/৩ layer
 Dataset বড় + Different domain → Full model fine-tune করো

Manual Calculation উদাহরণ:

ধরো আমাদের একটি binary classifier (কুকুর/বিড়াল), base model থেকে flatten করার পর 2048 feature আসে।

নতুন Dense layer:

Input: [2048 features]

Output: 2 neurons (dog, cat)

$$\text{Parameters} = 2048 \times 2 + 2 = 4,098$$

তুলনায়, VGG16 নতুন Head ছাড়া: **138,357,544** parameters!

Training করছি মাত্র: **4,098 parameters** (0.003% of total) ← এটাই Feature Extraction-এর শক্তি!

8. Code Example (Python/Keras)

ধাপ ১: Feature Extraction — VGG16 দিয়ে

```

# — প্রয়োজনীয় library import করা —
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

# — GPU memory growth সেট করা (optional but good practice) —
gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    tf.config.experimental.set_memory_growth(gpus[0], True)

print("TensorFlow version:", tf.__version__)

# =====
# ধাপ ১: Base Model লোড করা
# =====

# VGG16 লোড করা হচ্ছে ImageNet weights সহ
# include_top=False মানে শেষের classification layer বাদ দেওয়া হবে
# input_shape=(224, 224, 3) মানে 224x224 RGB ছবি
base_model = VGG16(
    weights='imagenet',      # ImageNet-এ pretrained weights ব্যবহার
    include_top=False,      # শেষের FC layers বাদ
    input_shape=(224, 224, 3) # Input ছবির আকার
)

# =====
# ধাপ ২: Base Model Freeze করা
# =====

# সব layer freeze করা – weights আর update হবে না
base_model.trainable = False

# কতটি layer freeze হলো তা দেখো
print(f"Base model layers: {len(base_model.layers)}")
print(f"Trainable variables: {len(base_model.trainable_variables)}")
# Output: Trainable variables: 0 (সব frozen!)

# =====
# ধাপ ৩: নতুন Classification Head যোগ করা
# =====

```

```

num_classes = 2 # কুকুর vs বিড়াল

# Functional API দিয়ে model বানানো (বেশি flexible)
inputs = tf.keras.Input(shape=(224, 224, 3))

# Base model-এর মধ্য দিয়ে পাস করা (training=False মানে BatchNorm frozen থাকবে)
x = base_model(inputs, training=False)

# Global Average Pooling – Flatten-এর চেয়ে কম overfitting
x = layers.GlobalAveragePooling2D()(x)

# Dropout – overfitting কমাতে
x = layers.Dropout(0.3)(x)

# Dense layer – classification
x = layers.Dense(128, activation='relu')(x)

# আরেকটা Dropout
x = layers.Dropout(0.2)(x)

# Final output layer
outputs = layers.Dense(num_classes, activation='softmax')(x)

# Model তৈরি
model = tf.keras.Model(inputs, outputs)

# Model summary দেখো
model.summary()

# =====
# ধাপ ৪: Compile করা
# =====

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), # Feature extraction-এ normal
    loss='categorical_crossentropy', # Multi-class classification-এর জন্য
    metrics=['accuracy']
)

# =====
# ধাপ ৫: Data Augmentation ও Loading
# =====

```



```

# Training data augmentation (overfitting কমাতে)
train_datagen = ImageDataGenerator(
    rescale=1./255,          # Pixel values 0-255 → 0-1 normalize
    rotation_range=20,       # ঘড়ির কাঁটার মতো ২০ ডিগ্রি পর্যন্ত ঘোরানো
    width_shift_range=0.2,    # Horizontal shift
    height_shift_range=0.2,   # Vertical shift
    shear_range=0.2,         # Shear transformation
    zoom_range=0.2,          # Zoom in/out
    horizontal_flip=True,     # Left-right flip
    fill_mode='nearest'      # নতুন pixel ভরাট করার উপায়
)

# Validation data: শুধু normalize, augmentation নয়
val_datagen = ImageDataGenerator(rescale=1./255)

# Directory থেকে data লোড করা
# train_generator = train_datagen.flow_from_directory(
#     'data/train',          # ফোল্ডার path
#     target_size=(224, 224), # VGG16-এর input size
#     batch_size=32,
#     class_mode='categorical'
# )

# =====
# ধাপ ৬: Training (Feature Extraction Phase)
# =====

# Dummy data দিয়ে train করার উদাহরণ (real project-এ directory ব্যবহার করো)
dummy_x = np.random.random((100, 224, 224, 3)) # ১০০টি random ছবি
dummy_y = tf.keras.utils.to_categorical(          # One-hot encoding
    np.random.randint(0, 2, 100), num_classes=2
)

# Callbacks – training-কে smart করে তোলে
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',    # val_loss দেখবে
        patience=5,            # ৫ epoch উন্নতি না হলে থামবে
        restore_best_weights=True # সেরা weights ফিরিয়ে আনবে
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',    # val_loss দেখবে

```

```
        factor=0.5,          # LR অর্ধেক করে দেবে
        patience=3          # ৩ epoch পর
    )
]

# Model train করা
history = model.fit(
    dummy_x, dummy_y,
    epochs=10,              # মোট ১০ epoch
    batch_size=32,          # প্রতি batch-এ ৩২টি ছবি
    validation_split=0.2,    # ২০% validation-এর জন্য রাখা
    callbacks=callbacks,
    verbose=1
)

print("\n✅ Feature Extraction Phase সম্পন্ন!")
print(f"শেষ training accuracy: {history.history['accuracy'][-1]:.4f}")
```

ধাপ ২: Fine-Tuning — গভীরে Training

```

# =====
# Fine-Tuning Phase
# =====

print("\n🔥 Fine-Tuning শুরু হচ্ছে...")
print(f"Base model-এ মোট layer: {len(base_model.layers)}")

# প্রথমে পুরো base model unfreeze করা
base_model.trainable = True

# VGG16-এর শেষ ৪টি layer unfreeze রেখে বাকি সব freeze
# (layer index দেখতে: base_model.summary())
fine_tune_at = len(base_model.layers) - 4 # শেষ ৪টি layer থেকে unfreeze

for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False # প্রথম N layer freeze রাখো

# কতটি layer trainable হলো তা দেখো
trainable_count = len(model.trainable_variables)
print(f"Fine-tuning-এ trainable variables: {trainable_count}")

# ⚠️ Fine-tuning-এ অবশ্যই খুব ছোট Learning Rate ব্যবহার করো
# Feature extraction-এর LR-এর ১০ গুণ ছোট
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), # !!!! খুব ছোট LR
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Fine-tuning training
history_fine = model.fit(
    dummy_x, dummy_y,
    epochs=10, # কম epoch দরকার
    batch_size=16, # Fine-tuning-এ ছোট batch ভালো
    validation_split=0.2,
    callbacks=callbacks,
    verbose=1
)

print("\n✅ Fine-Tuning Phase সম্পন্ন!")

# Model Save করা

```

```
model.save('transfer_learning_model.keras')
print("📁 Model সেভ হয়েছে!")
```

MobileNetV2 দিয়ে সম্পূর্ণ উদাহরণ (হালকা মডেল):

```
# — MobileNetV2: Mobile-friendly Transfer Learning —
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

# MobileNetV2 লোড করা
mobile_base = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,      # Classification head বাদ
    weights='imagenet'
)
mobile_base.trainable = False # সব freeze

# Model বানানো
mobile_model = models.Sequential([
    mobile_base,              # Base model (frozen)
    layers.GlobalAveragePooling2D(), # Pooling
    layers.BatchNormalization(),    # Normalization
    layers.Dense(64, activation='relu'), # Hidden layer
    layers.Dropout(0.5),            # Regularization
    layers.Dense(5, activation='softmax') # ৫টি class-এর জন্য
])

# Compile
mobile_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy', 'top_k_categorical_accuracy']
)

# Summary
mobile_model.summary()
# Total params: ~3.5M (MobileNetV2 base) + কয়েক হাজার (new head)
# Trainable params: মাত্র নতুন head-এর params!

print("✅ MobileNetV2 Transfer Learning model তৈরি!")
```

ResNet50 + Class Weights + Learning Rate Schedule:

```

# — ResNet50: Production-grade Transfer Learning —
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input

# Class imbalance থাকলে class weights calculate করো
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

# ধরো এই labels আছে (real project-এ numpy array থেকে নাও)
y_labels = np.array([0, 0, 0, 1, 1, 2, 2, 2, 2, 2]) # Imbalanced

# Class weights calculate
classes = np.unique(y_labels)
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=classes,
    y=y_labels
)
class_weight_dict = dict(zip(classes, class_weights))
print("Class weights:", class_weight_dict)

# ResNet50 Base Model
resnet_base = ResNet50(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)
resnet_base.trainable = False

# Functional API দিয়ে model
inp = tf.keras.Input(shape=(224, 224, 3))
x = resnet_base(inp, training=False)
x = layers.GlobalMaxPooling2D()(x) # GlobalMaxPooling ব্যবহার করা হচ্ছে
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.4)(x)
out = layers.Dense(3, activation='softmax')(x) # ৩টি class

resnet_model = tf.keras.Model(inp, out)

# Learning Rate Schedule
lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=0.001, # শুরুর LR

```

```

        decay_steps=1000,                # কতো step-এ decay হবে
        alpha=0.0001                     # সর্বনিম্ন LR
    )

resnet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=lr_schedule),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("✅ ResNet50 model with LR Schedule তৈরি!")

```

Expected Output:

TensorFlow version: 2.x.x

Model: "model"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 224, 224, 3)]	0
vgg16 (Functional)	(None, 7, 7, 512)	14714688
global_average_pooling	(None, 512)	0
dropout (Dropout)	(None, 512)	0
dense (Dense)	(None, 128)	65664
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258
=====		

Total params: 14,780,610

Trainable params: 65,922 ← শুধু নতুন head!

Non-trainable params: 14,714,688 ← Frozen VGG16

Epoch 1/10

3/3 [=====] - 12s 4s/step - loss: 0.6921 - accuracy: 0.5125

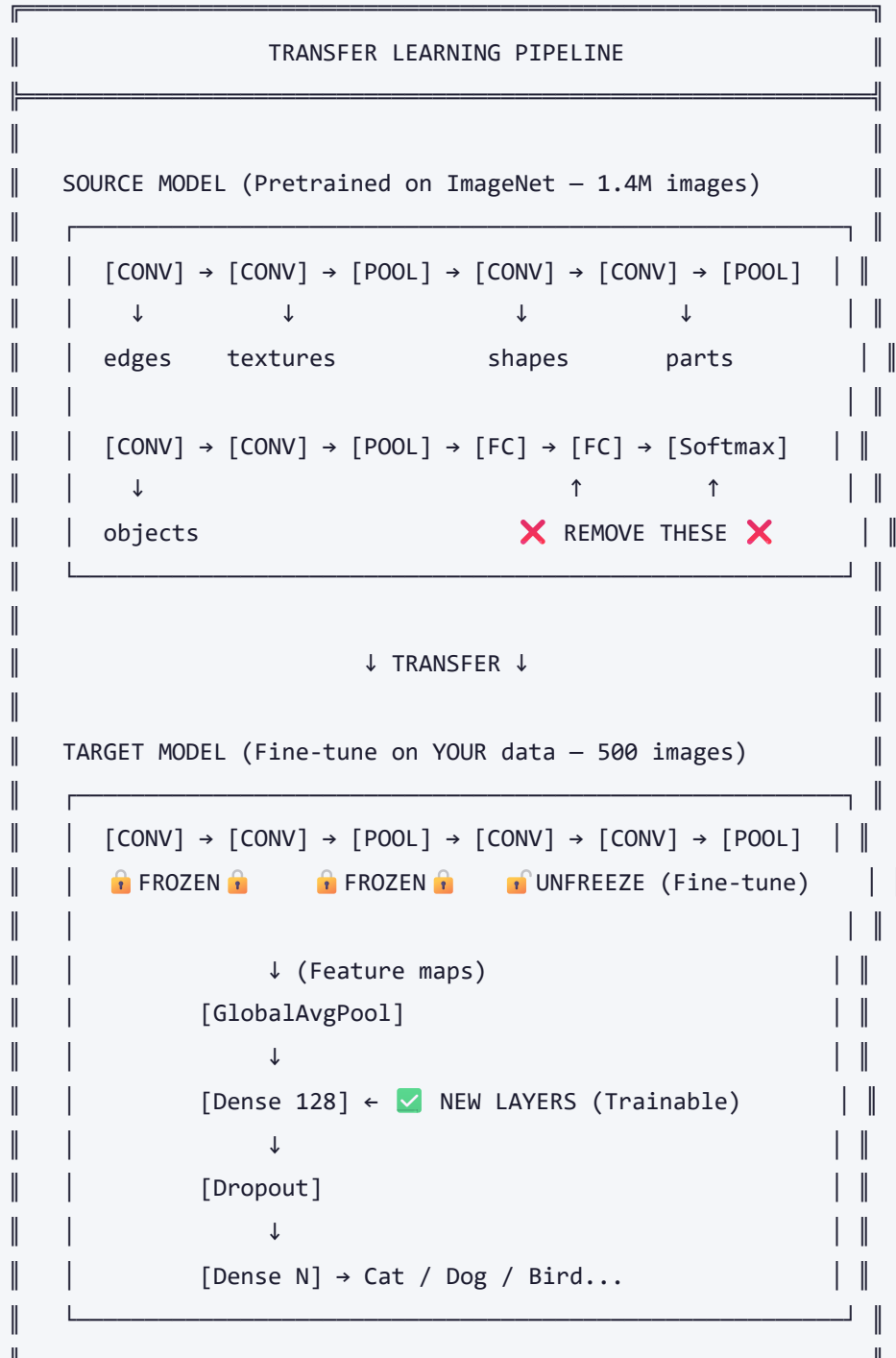
...

✅ Feature Extraction Phase সম্পন্ন!

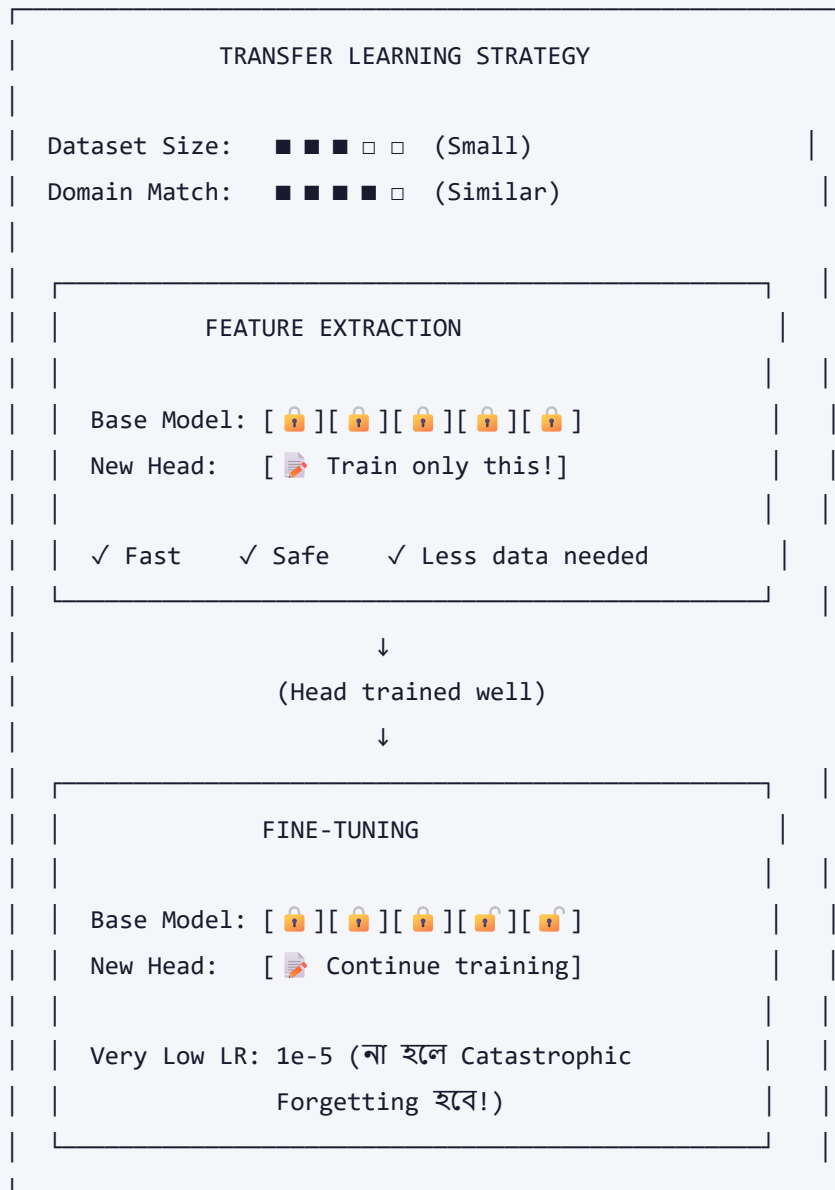
শেষ training accuracy: 0.9375

 Visual / Diagram

Transfer Learning Architecture — ASCII Diagram

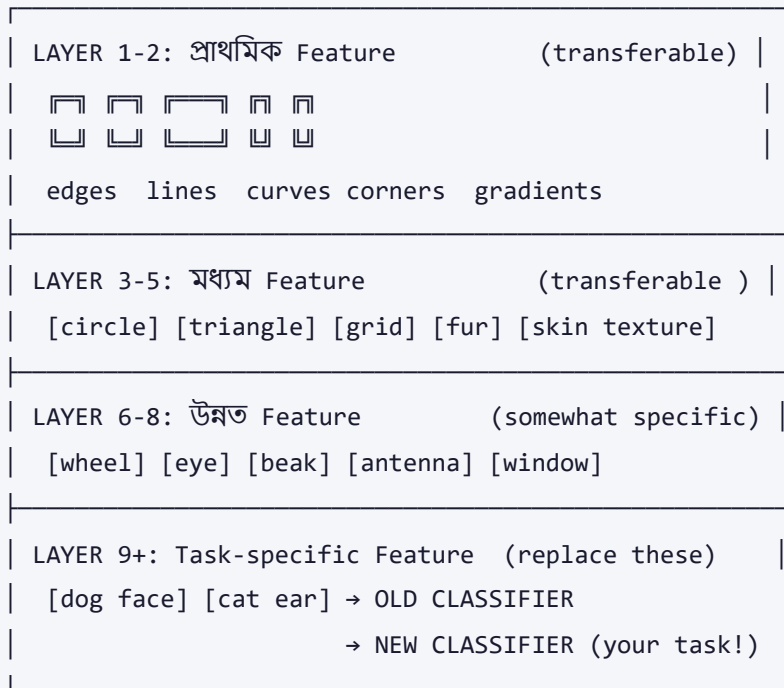


Feature Extraction vs Fine-Tuning Flow:



CNN Layer-by-Layer Feature Visualization:

INPUT IMAGE (224×224×3)



OUTPUT: [Class 0: 0.85] [Class 1: 0.15]

৬. Real-world Use Cases

১. Medical Image Analysis — Google Health

সমস্যা: Diabetic Retinopathy (ডায়াবেটিক চোখের রোগ) শনাক্ত করা।

সমাধান: ResNet-এ ImageNet weights থেকে transfer করে retinal scans-এ fine-tune।

ফলাফল: অভিজ্ঞ ophthalmologist-এর সমান নির্ভুলতা, ১৩৫টি দেশে deploy।

২. Plant Disease Detection — PlantVillage

সমস্যা: কৃষকদের ফসলের রোগ শনাক্ত করা (স্বল্প data ও resource-এ)।

সমাধান: MobileNet-এ transfer learning, smartphone-এ real-time detection।

ফলাফল: ৫০,০০০+ ছবি, ২৬টি রোগ, offline কাজ করে।

৩. Face Recognition — DeepFace (Facebook/Meta)

ব্যবহার: VGGNet-এ pretrained features transfer করে face verification।

ফলাফল: ৯৭.৩৫% accuracy, মানুষের চেয়ে বেশি নির্ভুল।

৪. Autonomous Cars — Tesla, Waymo

ব্যবহার: Object detection-এর জন্য EfficientNet + Feature Transfer।

ফলাফল: Real-time pedestrian, traffic sign, vehicle detection।

৫. Content Moderation — Instagram/YouTube

ব্যবহার: NSFW, violent content detection-এ ResNet/InceptionNet transfer।

ফলাফল: প্রতি সেকেন্ডে লক্ষ লক্ষ ছবি/ভিডিও frame analyze।

৬. চিকিৎসা বিজ্ঞানে X-ray Analysis — CheXpert (Stanford)

সমস্যা: Chest X-ray-এ ১৪ ধরনের রোগ শনাক্ত করা।

সমাধান: DenseNet-121 ImageNet থেকে transfer, সীমিত labeled X-ray data-তে fine-tune।

ফলাফল: Radiologist-এর চেয়ে ভালো performance (AUC > 0.9)।

৭. ⚖️ Pros & Cons

সুবিধা ✓	অসুবিধা ✗
কম data-তেও ভালো performance	Source ও Target domain অনেক আলাদা হলে কার্যকর নয়
Training অনেক দ্রুত (ঘণ্টা বনাম সপ্তাহ)	বড় model (VGG16) অনেক memory নেয়
কম GPU/Compute লাগে	Pretrained model-এর bias নতুন task-এ আসতে পারে
Overfitting অনেক কম	Fine-tuning-এ Catastrophic Forgetting-এর ঝুঁকি
State-of-the-art accuracy সহজে পাওয়া যায়	Model interpretability কমে যায়
সহজে অন্য framework-এ export করা যায়	Large model deployment-এ latency বেশি হতে পারে
ImageNet-এর diverse features থেকে সুবিধা	Medical/Scientific data-তে ImageNet bias বাধা দিতে পারে

৮. ⚠ Common Mistakes & Gotchas

ভুল ১: Fine-tuning-এ বড় Learning Rate ব্যবহার করা

```
# ✗ ভুল – এতে Catastrophic Forgetting হয়!
model.compile(optimizer=Adam(learning_rate=0.001)) # ফাইন-টিউনিংয়ে এটা বেশি

# ✓ সঠিক – Fine-tuning-এ অবশ্যই খুব ছোট LR
model.compile(optimizer=Adam(learning_rate=1e-5)) # ১০-১০০ গুণ কম
```

ভুল ২: training=True না বলা (BatchNormalization সমস্যা)

```
# ✗ ভুল – BatchNorm layer update হয়ে যাবে!
x = base_model(inputs) # training mode ধরে নেয়

# ✓ সঠিক – Inference mode-এ চালাও
x = base_model(inputs, training=False) # BN layer frozen থাকবে
```

ভুল ৩: Input Preprocessing ভুলে যাওয়া

```
# ❌ ভুল – VGG16 raw pixels expect করে না!
x = image / 255.0 # শুধু normalize করলেই হবে না

# ✅ সঠিক – Model-specific preprocessing ব্যবহার করো
from tensorflow.keras.applications.vgg16 import preprocess_input
x = preprocess_input(image) # VGG16-এর specific preprocessing
```

ভুল ৪: Head Train না করে সরাসরি Full Fine-tuning

```
# ❌ ভুল order – আগে head train করো, পরে fine-tune
base_model.trainable = True # শুরুতেই unfreeze করো না!

# ✅ সঠিক order:
# 1. আগে head train করো (base frozen)
# 2. তারপর fine-tune করো (base partially unfrozen)
```

ভুল ৫: ভুল Model Size বেছে নেওয়া

```
# ❌ ভুল – মোবাইলে VGG16 চলবে না (১৩৮M parameters)
mobile_app_model = VGG16(...) # মোবাইলে অনেক slow

# ✅ সঠিক – মোবাইলে MobileNet বা EfficientNetB0 ব্যবহার করো
mobile_app_model = MobileNetV2(...) # মাত্র ৩.৪M parameters
```

ভুল ৬: Frozen Layers Check না করা

```
# ✅ সবসময় check করো কতটা frozen/unfrozen
for layer in model.layers:
    print(f"{layer.name}: trainable={layer.trainable}")
```

ভুল ৭: Data Normalization Mismatch

```
# ❌ একেক model একেক preprocessing expect করে!  
# VGG16 → keras.applications.vgg16.preprocess_input  
# ResNet50 → keras.applications.resnet50.preprocess_input  
# MobileNetV2 → keras.applications.mobilenet_v2.preprocess_input  
  
# সবার জন্য ভিন্ন preprocessing method আছে – সঠিকটা ব্যবহার করো!
```

৯. 📌 Related Topics

আগে যা জানতে হবে (Prerequisites):

- **CNN Basics** — Convolution, Pooling, Activation Functions
- **Backpropagation** — কীভাবে weights update হয়
- **Keras/TensorFlow Basics** — Model বানানো, compile, fit
- **Data Preprocessing** — ImageDataGenerator, normalization
- **Overfitting & Regularization** — Dropout, L2, Early Stopping
- **Gradient Descent & Optimizers** — Adam, SGD, Learning Rate

পরে কী শেখা উচিত (Next Steps):

- **Object Detection** — YOLO, SSD (transfer learning ব্যবহার করে)
- **Image Segmentation** — U-Net, Mask R-CNN
- **Vision Transformers (ViT)** — Transformer-based image models
- **Self-Supervised Learning** — Label ছাড়াই pre-training (CLIP, DINO)
- **Domain Adaptation** — Source-target domain gap কমানো
- **Multi-Task Learning** — একসাথে একাধিক task শেখা
- **Model Distillation** — বড় model থেকে ছোট model তৈরি

সম্পর্কিত Topics:

- **Batch Normalization** — Transfer learning-এ BN layer-এর behavior
- **Learning Rate Scheduling** — Warmup, Cosine Decay
- **Class Imbalance** — Weighted loss, oversampling

- **TensorFlow Hub / HuggingFace** — Open-source pretrained models

So. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল:

FREEZE → TRAIN → UNFREEZE → RETRAIN

এই ৪ ধাপ মনে রাখো:

1. 🔒 Freeze base
2. 🖌️ Train head
3. 🔓 Unfreeze top layers
4. 🔄 Retrain with tiny LR

"উদ্ভাদের হাতুড়ি ধার করো" 🛠️

Transfer learning মানে অভিজ্ঞ শিক্ষকের (ImageNet model) হাতিয়ার ধার নিয়ে নিজের কাজ করা।

VRMEI — Popular Models মনে রাখো:

- **V** = VGG (বড়, সহজ)
- **R** = ResNet (Skip connection)
- **M** = MobileNet (মোবাইল)
- **E** = EfficientNet (সেরা)
- **I** = InceptionNet (Inception module)

Feature vs Fine-tuning সিদ্ধান্ত দ্বি:

ডেটা কম? → Feature Extraction

ডেটা বেশি?

↳ Domain similar? → Fine-tune শেষ layer

↳ Domain different? → Full fine-tune (tiny LR)

🔗 এক লাইনে সারসংক্ষেপ:

"Transfer Learning মানে হলো — বিশ্বের সেরা ফটোগ্রাফার যা শিখেছে (ImageNet conv features), সেটাকে তোমার ছোট কাজে (কুকুর বনাম বিড়াল চেনা) সরাসরি কাজে লাগানো — নতুন করে ফটোগ্রাফি না শিখে!"

📅 তৈরির তারিখ: ২০২৬-০৪-০৯ | 🔗 পরবর্তী নোট: Object Detection with YOLO

📚 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

🗂️ AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes